بِسْمِ اللهِ الرَّحْمنِ الرَّحِيمِ

Thursday, November 21, 2024 1:34 PM

Dear colleagues and friends,

As we immerse ourselves in the rich and intellectually stimulating field of Intelligent Systems, I find it fitting to reflect on a timeless piece of wisdom from Ali ibn Abi Talib:

"Wisdom enlivens the soul and guides the mind." [(1)]

This profound insight reminds us that our pursuit of knowledge is not just a technical endeavor but a transformative journey. In Intelligent Systems, we explore topics that lie at the very intersection of logic, reasoning, and innovation—concepts such as rational agents, search strategies, uncertainty, and machine learning. These areas challenge us to think deeply, solve complex problems, and ultimately contribute to a better understanding of how intelligence can be modeled and applied.

The lectures in this course—from the foundational principles of discrete mathematics and logical agents to the nuanced challenges of Bayesian networks and machine learning—serve as stepping stones toward creating systems that emulate human reasoning and decision-making. However, the true power of these systems lies in how we use them to benefit society, address real-world problems, and make informed, ethical decisions.

Let us approach this course with curiosity, diligence, and the wisdom to use what we learn for the greater good. By doing so, we not only enhance our skills but also contribute to building systems that reflect the best of human intelligence and morality.

_____

(1) Nahj al-Balagha (The Peak of Eloquence), Saying 58

Partial Exam 1

Tuesday, December 17, 2024 7:02 PM

L1: Introduction to Intelligent Systems and Discrete Mathematics
L2: State Spaces and Search Algorithms
L3: Informed Search
L4: Adversarial Search
L5: Logical Agents
L6: Propositional Logic 2
L7: Modeling Schnapsen with Propositional Logic, Rule-Based Reasoning, & Knowledge Graphs

L1: Introduction to Intelligent Systems and Discrete Mathematics

Monday, October 28, 2024 5:59 PM

Introduction

Intelligent Systems involve software and hardware that simulate intelligent behavior to perform tasks traditionally requiring human intelligence. These systems rely heavily on discrete mathematics—a branch of math focused on discrete elements that forms the backbone of algorithms and data structures used in Artificial Intelligence (AI). This overview will delve into core concepts such as problem-solving, knowledge representation, learning, and the mathematical foundations needed for AI.

Key Concepts in Intelligent Systems

1. Intelligent Systems Overview

- **Definition**: Intelligent Systems are systems that can sense, reason, learn, and act autonomously. They range from simple rule-based systems to complex adaptive AI systems.
- **Examples**:
  - **AlphaGo**: An AI developed by DeepMind that defeated a world champion in the strategy game Go. It uses deep learning and tree search algorithms to evaluate possible moves.
  - **Self-Driving Cars**: Autonomous vehicles that navigate roads using AI techniques to process sensor data and make real-time decisions.
  - **Voice Assistants**: AI like Siri or Google Assistant that interpret and respond to natural language commands using natural language processing (NLP).
- **Practical Application**: AI systems are deployed in various fields, such as healthcare for diagnosing diseases, finance for fraud detection, and retail for personalized product recommendations.

Discrete Mathematics for Intelligent Systems

Discrete Mathematics provides the theoretical foundation that makes AI and other computational systems possible. Here are the key areas and their significance:

2. Set Theory

- **Definition**: A set is a collection of distinct objects. Sets are fundamental for organizing and categorizing data.
- **Key Concepts**:
  - **Elements**: Objects contained in a set (e.g., {1, 2, 3}).
  - **Finite and Infinite Sets**: Finite sets have a limited number of elements (e.g., {1, 2, 3}), while infinite sets go on indefinitely (e.g., the set of natural numbers).
  - **Subsets**: A set **A** is a subset of **B** if all elements of **A** are also in **B** (e.g., {1, 2} ⊆ {1, 2, 3}).
  - **Power Set**: The set of all subsets of a set **S**. If S=a,bS = {a, b}S=a,b, then the power set P(S)=∅,a,b,a,bP(S) = {∅, {a}, {b}, {a, b}}P(S)=∅,a,b,a,b.
- **Practical Applications**:
  - In AI, sets help manage and categorize datasets, such as grouping data points for classification tasks.
  - **Database Management**: Set operations like unions, intersections, and differences are fundamental for querying databases.
- **Visualization**:
  - **Venn Diagrams**: Visual tools used to represent relationships between sets, useful for understanding overlaps and differences between categories.

3. Relations and Tuples

- **Definition**: Relations describe the connection between elements of two or more sets. A tuple is an ordered list of elements.
- **Key Concepts**:
  - **Ordered Pairs**: A pair of elements, such as (a,b)(a, b)(a,b), where order matters.

    - **n-Tuples**: An ordered list with n elements, e.g., a 3-tuple could be (x,y,z)(x, y, z)(x,y,z).
- **Practical Applications**:
    - **Databases**: Relational databases use relations to link data tables through common attributes, enabling complex data queries.
    - **Machine Learning**: Training data is often structured as tuples (feature, label) for supervised learning.

4. Graph Theory

- **Definition**: A graph is a mathematical structure used to model pairwise relations between objects. A graph consists of vertices (nodes) and edges (connections between nodes).
- **Types of Graphs**:
    - **Undirected Graph**: Edges have no direction, meaning the relationship between nodes is bidirectional.
    - **Directed Graph (Digraph)**: Edges have a direction, indicating a one-way relationship.
    - **Weighted Graph**: Edges have weights, representing costs or distances.
- **Key Concepts**:
    - **Adjacency Matrix**: A matrix used to represent a graph, where each cell indicates if an edge exists between nodes.
    - **Tree**: A special type of graph that is connected and acyclic (contains no loops).
- **Practical Applications**:
    - **Network Analysis**: Graphs are used to represent and analyze social networks, traffic routes, or the internet's web structure.
    - **Pathfinding Algorithms**: AI systems like GPS use graph algorithms to find the shortest route between locations.
- **Examples**:
    - **Social Networks**: Nodes represent individuals, and edges represent friendships.
    - **Logistics**: Nodes are warehouses, and edges are delivery routes with weights indicating distances.

Key Topics in Intelligent Systems

5. Problem Solving

- **Definition**: The process of identifying and executing the best solution among many possibilities. It involves algorithms that help navigate problem spaces.
- **Key Techniques**:
    - **Brute Force**: Exhaustively evaluating every possible option.
    - **Heuristics**: Rules-of-thumb that offer practical solutions without examining every possibility. Useful for large search spaces.
    - **Search Algorithms**:
        - **Depth-First Search (DFS)**: Explores as far as possible down each branch before backtracking.
        - **Breadth-First Search (BFS)**: Explores all nodes at the current depth before moving deeper.
- **Practical Applications**:
    - **Games**: AI in games like chess use search algorithms to evaluate future moves and choose optimal strategies.
    - **Operations Research**: Scheduling and resource allocation problems are solved using optimization algorithms.

6. Knowledge Representation and Reasoning

- **Definition**: The process of encoding information in a form that a computer can use to make decisions. It involves structuring knowledge to allow intelligent systems to infer new facts or make decisions.
- **Key Concepts**:
    - **Ontologies**: Frameworks that define categories and relationships between concepts, crucial for understanding domains.

    - **Logical Reasoning**: Using rules (if-then statements) to derive conclusions from known information.
    - **Knowledge Graphs**: Graph structures that represent knowledge, used in search engines and AI assistants.
- **Practical Applications**:
    - **Medical Diagnostics**: AI systems diagnose diseases by reasoning about symptoms using a database of medical knowledge.
    - **Virtual Assistants**: Voice assistants use structured information to interpret user queries and provide relevant answers.

7. Learning and Adaptivity

- **Definition**: Learning refers to the ability of systems to improve their performance over time, based on experience or data. Machine learning, a subset of AI, enables systems to recognize patterns and make predictions.
- **Types of Learning**:
    - **Supervised Learning**: Learning from labeled data (e.g., email classified as spam or not).
    - **Unsupervised Learning**: Finding patterns in data without pre-existing labels (e.g., clustering).
    - **Reinforcement Learning**: Learning optimal actions through trial and error (e.g., training an agent to play a game).
- **Practical Applications**:
    - **Computer Vision**: Systems learn to recognize objects in images using deep learning.
    - **Recommendation Systems**: Algorithms suggest products or content based on past behavior.

L2: State Spaces and Search Algorithms

Thursday, November 21, 2024 11:12 AM

Introduction

This lecture introduces state-space representations and uninformed search algorithms, key tools in artificial intelligence for decision-making. The focus is on:

- How agents operate in different environments.
- Representing real-world problems using state spaces.
- Exploring algorithms like breadth-first search (BFS) and depth-first search (DFS) to navigate these representations.

Key Sections and Concepts

1. Agents and Decision-Making

- **Definition of Agents**:
  - An agent perceives its environment through sensors and manipulates it using actuators.
  - Levels of abstraction:
    1. **Functional**: Behavior as a function mapping sensor inputs to actions.
    2. **Programmatic**: Implementation within a system.
    3. **Architectural**: Underlying operational infrastructure.
- **Types of Agents**:
  - **Simple Reflex Agents**: Rely on condition-action rules without internal state.
  - **Model-Based Reflex Agents**: Maintain an internal state to handle partially observable environments.
  - **Goal-Based Agents**: Optimize actions to achieve specific future goals.
- **Example**: A room-heating agent versus an advanced robot like Grace, showcasing complexity differences.

2. State-Space Representations

- **Definition**: A mathematical model where nodes represent states, and edges represent actions.
- **Purpose**:
  - Abstract real environments to simplify problem-solving.
  - Enable the use of generic algorithms to address common decision-making challenges.
- **Examples**:
  - Robotic vacuum cleaners using simplified maps.
  - Graph representations for geographical problems (e.g., Romanian cities).
- **PEAS Framework**: Performance, Environment, Actuators, Sensors criteria help define agent environments.

3. Search Algorithms

- **Purpose**: Navigate state-space graphs efficiently to find solutions.
- **Tree Search Process**:
  1. Initialize with the root node (initial state).
  2. Expand nodes using a successor function until reaching the goal or exhausting possibilities.
- **Key Algorithms**:
  1. **Breadth-First Search (BFS)**:
     - Explores all nodes at one depth level before moving deeper.
     - **Advantages**: Completeness, optimality.
     - **Challenges**: High space complexity due to the large number of stored nodes.
  2. **Depth-First Search (DFS)**:

        - Explores as deep as possible along one branch before backtracking.
        - **Advantages**: Lower space complexity.
        - **Challenges**: Vulnerability to loops and non-optimal solutions.
    3. **Depth-Limited Search (DLS)**:
        - Sets a depth limit to mitigate DFS's infinite loop issue.
    4. **Iterative Deepening Search (IDS)**:
        - Combines DFS's space efficiency with BFS's systematic search, iterating through increasing depth levels.

Critical Insights

- **Metrics for Evaluating Search Algorithms**:
    - **Completeness**: Can it always find a solution?
    - **Optimality**: Does it find the least-cost solution?
    - **Time Complexity**: Number of nodes expanded.
    - **Space Complexity**: Memory required for node storage.
- **Real-World Applications**:
    - Robotic systems (e.g., vacuum cleaners).
    - Puzzle-solving.
    - Pathfinding in dynamic environments.
- **Understanding PEAS** helps design effective agents.

Glossary of Terms

1. **State-Space**: A representation of possible configurations in a problem domain.
    - Example: Chessboard states.
2. **PEAS**: Framework for defining task environments (Performance, Environment, Actuators, Sensors).
3. **Fringe**: Data structure used in search algorithms to store nodes for exploration.

Discussion Questions

1. Which search algorithm guarantees an optimal solution in a graph with uniform path costs?
    - A) BFS
    - B) DFS
    - C) IDS
    - D) DLS
2. What is a key disadvantage of DFS?
    - A) High space complexity
    - B) Incompleteness due to loops
    - C) High time complexity
    - D) Requirement for a heuristic

---

1. **Answer**: A) BFS, because it systematically explores levels.
2. **Answer**: B) Incompleteness due to loops.

L3: Informed Search

Thursday, November 21, 2024 11:14 AM

Introduction

This lecture focuses on **informed search algorithms**, contrasting them with uninformed search methods covered previously. Informed search introduces heuristics to improve the efficiency and optimality of search processes. The primary objective is to leverage domain-specific knowledge to guide searches intelligently, often reducing time and memory requirements.

Key Concepts and Recap

1. **Uninformed Search Algorithms**:
    - **Breadth-First Search (BFS)**: Explores all nodes at the current depth before moving deeper.
    - **Depth-First Search (DFS)**: Explores as far as possible along one path before backtracking.
    - **Iterative Deepening Search (IDS)**: Combines the benefits of BFS and DFS by incrementally increasing depth limits.
      Performance Metrics:
    - Completeness
    - Optimality
    - Time complexity
    - Space complexity
      Shortcomings:
    - Often computationally expensive.
    - Limited by lack of domain-specific knowledge.
2. **Goal-Driven vs. Data-Driven Approaches**:
    - **Data-Driven**: Starts from the initial state and moves towards the goal (e.g., family tree analysis with a high branching factor).
    - **Goal-Driven**: Starts from the goal and reasons backward, often preferable when branching factors are lower in the reverse direction.

Informed Search Algorithms

1. **Introduction to Heuristics**:
    - Heuristics are domain-specific "rules of thumb" to estimate the cost to reach the goal.
    - Common heuristics:
        - **Manhattan Distance**: Sum of horizontal and vertical distances.
        - **Straight-Line Distance (SLD)**: Direct distance, often optimistic.
2. **Best-First Search**:
    - Expands the most promising node based on heuristic h(n).
    - Example: Sorting nodes by their heuristic values (e.g., navigating Romanian cities using SLD).
      Challenges:
    - Prone to loops and inefficiency due to backtracking issues.
3. **Greedy Best-First Search**:
    - Chooses nodes with the lowest heuristic value without considering cumulative path costs.
    - Vulnerabilities:
        - Incomplete due to loops.
        - Non-optimal when the heuristic is misleading.
4. **Beam Search and Hill Climbing**:
    - **Beam Search**: Limits the number of expanded nodes to k, sacrificing optimality for efficiency.
    - **Hill Climbing**: Moves towards the local optimum but risks getting stuck at local maxima.
5. *A and A Search**:

- **Evaluation Function**: f(n)=g(n)+h(n)
- g(n)): Cost so far.
  - h(n): Estimated cost to goal.
- **Admissibility and Monotonicity**: Ensures heuristics never overestimate costs.
- Advantages:
  - Optimal and complete with admissible heuristics.
- Disadvantages:
  - High computational cost.

Critical Insights

- **Heuristic Design**:
  - The choice of heuristic significantly impacts the algorithm's efficiency and effectiveness.
- **Trade-offs**:
  - Efficiency (e.g., beam search) vs. completeness and optimality (e.g., A*).
- **Practical Applications**:
  - Puzzle solving, route optimization, and AI in games (e.g., chess).

Key Takeaways

- Informed search introduces heuristics to guide the search efficiently.
- A* is a gold standard for its optimality but at a computational cost.
- Algorithms must be chosen based on the problem's requirements, such as optimality, efficiency, or memory constraints.

Practice Quiz

1. **Which of the following is a property of admissible heuristics?**
   - (a) Overestimates the cost to the goal.
   - (b) Guarantees an optimal solution if one exists.
   - (c) Never uses memory.
   - (d) Relies only on BFS.

     Explanation: Admissible heuristics never overestimate the cost, ensuring the solution is optimal.
2. **What is the primary drawback of greedy best-first search?**
   - (a) Completeness
   - (b) Optimality
   - (c) Memory usage
   - (d) Simplicity

---

1. B
2. B

Study Tips

1. **Mnemonic for A***: Remember "A star is optimal but expensive."
2. **Flowchart**: Visualize differences in search strategies (e.g., BFS vs. DFS vs. A*).
3. **Compare Heuristics**: Practice calculating Manhattan and SLD for simple problems.

Glossary

- **Heuristic**: Rule for estimating the cost of reaching a goal.
  - *Example*: Manhattan distance in grid problems.
- **Admissibility**: Property of never overestimating the cost to the goal.
  - *Example*: SLD is admissible.

L4: Adversarial Search

Thursday, November 21, 2024 12:25 PM

Introduction

Adversarial search is a fundamental topic in artificial intelligence (AI), focusing on environments with multiple agents competing to achieve their goals. This lecture emphasizes adversarial search methods, particularly the Minimax algorithm and its optimization through alpha-beta pruning. Applications in games like tic-tac-toe, chess, and complex games like Go are discussed, along with challenges posed by imperfect information.

Key Sections

1. **Agents and Environments**
   - Agents interact with environments, which may provide full or partial information.
   - **Partial Information:**
     - Agents must reason under uncertainty, relying on belief states (e.g., Vacuum World example where rooms may or may not be clean).
     - Sensorless problems reduce manufacturing costs but often result in inefficiencies.
2. **Games as Adversarial Environments**
   - Games provide a structured environment to study adversarial search due to clear rules.
   - Two classifications:
     - **Deterministic vs. Stochastic:** Games may depend solely on players' actions (deterministic) or involve randomness (stochastic, e.g., dice rolls).
     - **Perfect vs. Imperfect Information:** Perfect information implies full visibility of the game state (e.g., chess), while imperfect information involves hidden elements (e.g., Schnapsen).
3. **Search in Adversarial Settings**
   - Search methods for adversarial environments differ from standard search algorithms due to the presence of opponents.
   - A game's components analogized with search:
     - Initial state = Starting game position.
     - Successor functions = Possible moves.
     - Terminal test = Endgame conditions.
     - Utility function = Value of terminal states (+1 win, -1 loss, 0 draw).
4. **The Minimax Algorithm**
   - **Definition:** A decision-making algorithm where one agent (MAX) aims to maximize its score, while the other (MIN) minimizes it.
   - **Steps:**
     - Explore the game tree from terminal nodes upwards.
     - Alternate between maximizing and minimizing scores based on whose turn it is.
   - **Properties:**
     - Complete and optimal under assumptions of rational play.
     - Computational complexity grows exponentially with depth, limiting practicality for complex games like chess.
5. **Alpha-Beta Pruning**
   - **Purpose:** Reduce the number of nodes evaluated in Minimax by pruning irrelevant branches.
   - **Mechanism:**
     - Two parameters, alpha (MAX's best option) and beta (MIN's best option), are maintained.
     - Pruning occurs when alpha exceeds beta, as further exploration will not affect the

decision.
    - **Benefits:**
        - Halves time complexity, allowing deeper exploration (e.g., from 6 to 12 moves in chess).
6. **Heuristic and Utility Evaluation**
    - **Utility Functions:** Provide exact state evaluations (e.g., game outcomes).
    - **Heuristics:** Offer approximations to guide decision-making when exhaustive evaluation is infeasible.
7. **Imperfect Information and Advanced Techniques**
    - Challenges in games like Schnapsen arise from imperfect information and combinatorial explosion.
    - Solutions:
        - **Monte Carlo Sampling:** Selects a subset of scenarios for evaluation.
        - **Random Rollouts:** Simulates games randomly to approximate win probabilities.
8. **Case Study: AlphaGo**
    - Combines Minimax, alpha-beta pruning, and Monte Carlo tree search.
    - Demonstrated AI's potential by defeating a top Go player, highlighting advancements in adversarial search.

Key Takeaways

- Adversarial search requires agents to account for opponents' actions, making decision-making more complex than in single-agent environments.
- Minimax is foundational but computationally intensive; alpha-beta pruning mitigates this by reducing redundant calculations.
- Heuristics and sampling techniques address challenges in games with high complexity or imperfect information.
- Applications extend beyond games, influencing AI's approach to strategic decision-making.

Suggested Discussion Questions

1. **Multiple Choice Questions (MCQs):**
    - **What does the alpha parameter in alpha-beta pruning represent?**
        - (A) The maximum value found by MIN
        - (B) The maximum value found by MAX
        - (C) The minimum value found by MAX
        - (D) None of the above
    - **Which of the following is NOT a property of the Minimax algorithm?**
        - (A) Completeness
        - (B) Exponential time complexity
        - (C) Suboptimality
        - (D) Rational decision-making

_____

1. **Correct Answer:** (B) The maximum value found by MAX
   Explanation: Alpha tracks the best choice MAX can ensure at a given point.
2. **Correct Answer:** (C) Suboptimality
   Explanation: Minimax is optimal under rational play assumptions.

_____

Study Tips

- Visualize algorithms using game trees and diagrams to understand decision-making processes.
- Use mnemonics to recall steps in Minimax and conditions for alpha-beta pruning (e.g., “Alpha

trims Above, Beta trims Below”).

- Practice constructing small game trees to manually apply Minimax and alpha-beta pruning.

Glossary

- **Minimax Algorithm:** Decision rule for minimizing the possible loss in a worst-case scenario.
- **Alpha-Beta Pruning:** Optimization technique for reducing unnecessary calculations in Minimax.
- **Heuristic Evaluation:** Approximation method for assessing the desirability of states.
- **Monte Carlo Sampling:** Statistical method for approximating outcomes through random sampling.

L5: Logical Agents

Thursday, November 21, 2024 12:26 PM

Introduction

The lecture introduces the use of **logic in artificial intelligence (AI)**, specifically for expressing and manipulating information under **imperfect conditions**. Logic plays a key role in structured problem-solving, game theory, and AI environments.

Detailed Breakdown

1. Logic in Imperfect Information

- **Definition**: Logic helps represent knowledge and make decisions even when all information is not available.
- **Example: Schnapsen Card Game**:
  - **Phase 1**: Uses the **minimax algorithm** to evaluate possible outcomes.
  - **Challenges**: Impossible to calculate all deals due to high complexity.
  - **Solution**: Use **Perfect Information Monte Carlo Sampling (PIMS)** to reduce complexity by sampling a subset of scenarios.
  - **Phase 2**: Implements random rollouts to estimate win probabilities.
- **Application to AI Games**:
  - Google DeepMind's **AlphaGo** combines minimax, **Monte Carlo Tree Search**, and **random rollouts** to outperform human players in Go, a game with immense search space.

2. Wumpus World

- **Scenario**: An agent navigates a grid to find gold while avoiding pits and the Wumpus.
- **Environment Properties (PEAS)**:
  - **Partial Observability**: The agent relies on percepts like "stench" (Wumpus nearby) or "breeze" (pit nearby).
  - **Logic in Action**:
    - **Formal Knowledge Base (KB)**: Stores rules and percepts.
    - **Propositional Logic**: Determines safe moves.
    - **Syntax and Semantics**: Allow encoding of environment states.
- **Key Insights**:
  - Logical reasoning reduces uncertainty in imperfectly observable environments.
  - Propositions like ¬B1,1 (no breeze in square [1,1]) guide decisions.

3. Propositional Logic

- **Formal System Components**:
  - **Syntax**: Rules for writing formulae (e.g., operators like ¬, ∧, ∨, →, ⇔).
  - **Semantics**: Meaning of formulae, evaluated as true (1) or false (0).
  - **Calculus**: Rules for manipulating formulae to infer truth.
- **Truth Tables**:
  - Used to evaluate complex logical statements.
  - Example: $B_{1,1} \Leftrightarrow (P_{1,2} \vee P_{2,1})$
- **Operators**:
  - **Negation (¬)**: Reverses truth value.
  - **Conjunction (∧)**: True if both propositions are true.
  - **Disjunction (∨)**: True if at least one proposition is true.
  - **Implication (→)**: False only when the antecedent is true, and the consequent is false.

- **Biconditional (⇔)**: True when both propositions have the same truth value.

4. Knowledge Bases and Models

- **Knowledge Base (KB)**: A collection of true statements.
  - **Satisfiability**: A KB is satisfiable if at least one model exists.
  - **Validity**: A sentence is valid if it is always true.
  - **Entailment**: A KB entails a sentence if the sentence is true in all models of the KB.
- **SAT Solving**:
  - Determines satisfiability of complex logical systems.
  - Example: Applied to Wumpus World for finding valid states.

Key Takeaways

1. Logic structures human knowledge into systems AI can manipulate.
2. Imperfect information requires sampling techniques like PIMS.
3. Propositional logic is foundational in reasoning and decision-making.
4. Knowledge bases use models to infer truth, supporting SAT solving.

Discussion Questions

1. **What is the role of sampling in dealing with imperfect information?**
   - A. It reduces computational complexity.
   - B. It ensures all scenarios are calculated.
   - C. It avoids using logic operators.
2. **Which operator is true only when both propositions are true?**
   - A. ∨
   - B. ∧
   - C. →
3. **How does a truth table help in propositional logic?**
   - A. By reducing syntax errors.
   - B. By visually representing all truth value combinations.
   - C. By verifying models of invalid statements.

---

1. **Correct Answer**: A. Sampling simplifies decision-making by evaluating a manageable subset of scenarios.
2. **Correct Answer**: B. The conjunction operator (∧) is true only if both connected propositions are true.
3. **Correct Answer**: B. Truth tables illustrate all possible truth combinations, aiding logical evaluations.

Study Tips

- **Visual Aids**: Draw truth tables and flowcharts for practice.

Glossary

- **Propositional Logic**: A language for evaluating truth in statements.
  - **Example**: $B_{1,1} \Leftrightarrow (P_{1,2} \vee P_{2,1})$
- **Knowledge Base**: Collection of statements known to be true.
- **SAT Solving**: Process of determining if a logical formula is satisfiable.

L6: Propositional Logic 2

Thursday, November 21, 2024 12:28 PM

Introduction

This lecture builds on foundational concepts in propositional logic, focusing on the satisfiability of sentences, entailment in knowledge bases, and practical methods for problem-solving, such as the Davis-Putnam-Logemann-Loveland (DPLL) algorithm. The lecture introduces Clause Normal Form (CNF) as a crucial notation for computational logic and examines its applications in SAT solving, with examples like the Wumpus World problem.

Detailed Breakdown

Definitions and Key Concepts

1. **Satisfiability**:
   - A sentence is **satisfiable** if a model can be found that makes the sentence true.
   - A sentence is **unsatisfiable** if no such model exists.
2. **Validity**:
   - A sentence is **valid** if it is true under all possible truth assignments.
3. **Entailment**:
   - A sentence is entailed by a knowledge base if it is true in all models of the knowledge base.
   - Example: In a model where B must be true for the model to be valid, B is entailed.

SAT Solving Example: Wumpus World

- Models are generated based on the observations and rules of the game.
- Only states that align with these constraints are considered valid models.

Clause Normal Form (CNF)

1. **Definition**:
   - CNF is a standardized way to write logical sentences, making them easier to process computationally.
2. **Notation**:
   - Uses sets and commas instead of conjunction symbols (∧).
   - Example: ((¬A ∨ B) ∧ (¬B ∨ A) ∧ C) can be represented as {¬A ∨ B}, {¬B ∨ A}, {C}.
3. **Applications**:
   - Input for algorithms like DPLL for satisfiability testing.

DPLL Algorithm

1. **Purpose**:
   - A procedure for determining the satisfiability of a logical formula.
2. **Steps**:
   - Starts with no assignments and incrementally assigns values to propositions.
   - If a partial assignment leads to a false sentence, it backtracks.
3. **Variants**:
   - **Variant 1**:
     - Begins with P = 0.
     - Uses unit clauses to simplify the formula.
     - Backtracks as necessary and finds a solution model.
   - **Variant 2**:
     - Prioritizes variables that occur most frequently in the smallest clauses.
     - Quickly simplifies the formula to find a solution.
   - **Variant 3**:
     - Similar to Variant 2 but prioritizes variables with more positive occurrences.

Critical Insights and Applications

1. **CNF** simplifies satisfiability testing by structuring logical statements in a computationally tractable format.
2. **DPLL Algorithm** provides a systematic method for solving SAT problems and is foundational in AI problem-solving frameworks.

Bullet-Point Takeaways

- A valid sentence is always true, while a satisfiable sentence is true in at least one model.
- Entailment occurs when a sentence is true in all models of a knowledge base.
- CNF notation is pivotal for computational applications like SAT solving.
- DPLL is an efficient algorithm for satisfiability problems, with variations enhancing performance based on problem structure.

Suggested Practice Questions

1. **What is the difference between satisfiability and validity?**
    - a) Satisfiability applies to all models, validity applies to some models.
    - b) Satisfiability applies to some models, validity applies to all models.
    - c) Both terms mean the same.
    - d) None of the above.
2. **Why is CNF important in computational logic?**
    - a) It simplifies logical expressions for human readability.
    - b) It allows algorithms like DPLL to process formulas efficiently.
    - c) It is a subset of predicate logic.
    - d) None of the above.
3. **What is the key advantage of Variant 2 of the DPLL algorithm?**
    - a) It minimizes the use of unit clauses.
    - b) It prioritizes the variable occurring most often in small clauses.
    - c) It avoids backtracking altogether.
    - d) None of the above.

---

1. **Answer**: b. Satisfiability requires at least one model, while validity requires all models.
2. **Answer**: b. CNF standardizes formulas for algorithmic processing.
3. **Answer**: b. Variant 2 optimizes the process by prioritizing frequently occurring variables.

Tips for Remembering Key Concepts

1. **Use acronyms**:
    - "SEV" for **Satisfiability, Entailment, Validity**.
2. **Visual aids**:
    - Draw truth tables or flowcharts for CNF transformations.
3. **Chunking**:
    - Group related concepts (e.g., CNF and DPLL) for better recall.

Glossary

- **Satisfiable**: A logical sentence with at least one true model (e.g., $A \vee B$ is satisfiable if A = 1 or B =1).
- **Entailment**: When a sentence is true across all models of a knowledge base.
- **Clause Normal Form (CNF)**: A standardized logical representation using conjunctions of disjunctions.
- **DPLL Algorithm**: A SAT-solving procedure that uses partial assignments and backtracking.

L7: Modeling Schnapsen with Propositional Logic, Rule-Based Reasoning, & Knowledge Graphs

Thursday, November 21, 2024 12:29 PM

Introduction
This lecture focuses on the application of artificial intelligence techniques to model and reason about structured systems, such as the card game Schnapsen and knowledge graphs. It explores the utility of propositional and first-order logic, rule-based reasoning, and knowledge graphs in building intelligent systems.

Part 1: Propositional Logic for Schnapsen (not for exam)

- **Logic for Games**:
  - **Purpose**: Uses logic to analyze game states and devise strategies.
  - **Example**: Determining safe moves using propositional formulas.
- **Reasoning Over Models**:
  - Encodes card states and colors as propositional variables.
  - Incorporates imperfect information to simulate human-like reasoning.
- **Modeling Strategies**:
  - Strategies like PlayNoTrumpJack and TrumpWedding are encoded in propositional and first-order logic.
- **Pros and Cons**:
  - **Pros**: Declarative, compositional, and handles partial/negated information.
  - **Cons**: Limited expressiveness, especially for complex or infinite domains.

Part 2: Rule-Based Reasoning

- **Inference Rules**:
  - Defines rules to derive conclusions from premises.
  - Includes techniques like **Modus Ponens** and logical equivalence.
- **Proof Searching**:
  - Explores monotonic reasoning where entailed knowledge grows with new data.
  - Techniques include **Forward Chaining** (data-driven) and **Backward Chaining** (goal-driven).
- **Calculus for Forward Reasoning**:
  - Establishes soundness and completeness for forward chaining in propositional logic.

Part 3: Knowledge Graphs

- **Purpose**:
  - Facilitates associations between objects in AI and the Semantic Web.
- **Ontologies**:
  - **Definition**: An explicit specification of shared conceptualizations within a domain.
  - Components include instances, classes, attributes, and relations.
- **Knowledge Representation**:
  - Uses logical formulas to encode relations (e.g., Amsterdam is a city).
- **Triple Syntax**:
  - RDF and RDF Schema (RDFS) vocabulary models knowledge using "subject-predicate-object" triples.
  - Examples:

        - Class membership: ex: Netherlands rdf:type ex:EuropeanCountry
        - Subclass relations: ex:EuropeanCountry rdfs:subClassOf ex:Country.

Part 3.2: Knowledge Engineering

- **Process**:
    1. Define domain and scope.
    2. Enumerate terms and develop taxonomy.
    3. Define properties and classes.
    4. Create instances and test for anomalies.
- **Example**:
    - A card game ontology where cards have attributes (e.g., Jack hasValue 2).

Part 4: Tools (not for exam)

- **SPARQL**:
    - A query language to retrieve structured data from knowledge graphs.
    - Examples provided for Python integration.
- **Entailment and Reasoning**:
    - Implements reasoning using tools like owlrl for RDF and RDFS entailments.

Conclusion

Propositional and first-order logic provide powerful but sometimes limited tools for modeling. Rule-based reasoning complements logic for inference, while knowledge graphs offer scalable solutions for structuring and querying complex systems.

Key Takeaways

- Logic enables structured reasoning for games but has limitations in complexity and domain scope.
- Rule-based reasoning offers a method for deriving knowledge but requires careful rule design.
- Knowledge graphs provide scalable, structured representations essential for semantic reasoning.

Discussion Questions (Multiple-Choice)

1. **How does RDF represent relationships in knowledge graphs?**
    - A) Using logical operators
    - B) Through subject-predicate-object triples
    - C) By inference rules

---

1. **Correct Answer**: B) Through subject-predicate-object triples (Explanation: RDF uses triples for structured data representation).

Glossary

- **Propositional Logic**: A branch of logic dealing with declarative sentences that are either true or false.
- **First-Order Logic**: Extends propositional logic to include quantifiers and relations.
- **Knowledge Graph**: A graph-based representation of structured knowledge using nodes (entities) and edges (relations).
- **Ontology**: Formal representation of a set of concepts within a domain and their relationships.
- **SPARQL**: Query language for retrieving and manipulating RDF data.

Partial Exam 2

Tuesday, December 17, 2024 7:02 PM

L8: Vagueness and Uncertain Knowledge
L9: Uncertainty (Probabilities) and Bayesian Networks
L10: Machine Learning 1
L11: Machine Learning 2
L12: Machine Learning 3
L13: Machine Learning 4

L8: Vagueness and Uncertain Knowledge

Tuesday, December 17, 2024 7:05 PM

Introduction to Vagueness and Uncertainty

This lecture explores the concepts of vagueness and uncertainty in AI and introduces the formalisms used to address these concepts, such as **fuzzy sets** and **probability**. While these terms are closely related, they serve distinct purposes:

- **Vagueness** refers to imprecise or unclear boundaries in information.
- **Uncertainty** deals with partial or unreliable observability of information.

Key Concepts

1. Difference Between Vagueness and Uncertainty

- **Vagueness**: The concept of a glass being “half full” is vague because the notion of “half” is not precise. Fuzzy logic can represent this as “half full to a high degree.”
- **Uncertainty**: If the room becomes dark, and the glass is no longer observable, there is uncertainty due to lack of information.

Vagueness	Uncertainty
Deals with imprecise information	Deals with missing or incomplete data
Example: “Nearly full” glass	Example: “How much water is there?”

2. Fuzzy Sets and Membership Functions

- Traditional propositional logic restricts statements to **True** or **False**.
- In contrast, **fuzzy sets** allow a **degree of membership** between 0 and 1.
    - Example: A person 170 cm tall is "tall" to a certain degree compared to a person 120 cm tall.
- Membership values are **normalized** for comparison (0 to 1).

Fuzzy Membership Shapes

- Linear or curved functions can model vagueness:
    - Example: “Nearly half-full” can have a triangular shape where the degree is highest at the midpoint.

3. Dealing with Uncertainty

- **Uncertainty** arises due to unreliable, incomplete, or imprecise data.
- Example of uncertainty: An automated taxi faces unknown factors like traffic conditions, road closures, and other agents' behavior.

Logic in Uncertain Environments

- Early AI systems used **rule-based expert systems** with declarative rules.
- Example: “If I leave 25 minutes early, I arrive on time 30% of the time.”
- The challenge: Combining multiple rules can be inconsistent.

4. Probability Theory

Probability provides a mathematical way to handle uncertainty. Two main paradigms exist:

- **Frequentism**:
    - Probability is based on observed frequencies in a population.
    - Example: If 400 out of 1000 students pass, the next batch is expected to have a 40% pass rate.
- **Bayesianism**:
    - Probability reflects subjective belief or uncertainty about a single event.
    - Example: Bob has a 20% chance of an event occurring, but the event either occurs or it

doesn't.

Key Probability Terms

1. **Sample Space (Ω)**: All possible outcomes of an experiment.
   - Example: Coin toss → {Heads, Tails}.
2. **Atomic Events**: Single elements in the sample space.
3. **Event**: A combination of atomic events.
   - Example: Rolling a dice → {Outcome < 5}.
4. **Probability Value (P)**:
   - Ranges between 0 (impossible) and 1 (certain).
   - Sum of all atomic events = 1.

Random Variables

- A **random variable** assigns a value (true/false, discrete, continuous) to atomic events.
- Example:
  - Odd(1) = True, Odd(2) = False.

Key Takeaways

- Vagueness is handled using fuzzy logic, while uncertainty uses probability.
- Fuzzy sets allow degrees of membership, whereas probability deals with likelihoods of events.
- Two paradigms of probability exist: frequentism and Bayesianism.
- Logic frameworks are essential for reasoning under uncertainty.

Practical Applications

- **Fuzzy Logic**: Used in control systems, such as air conditioners and washing machines, to handle vague inputs like "warm" or "cool."
- **Probabilities**: Applied in AI systems for predictions, robotics (e.g., autonomous taxis), and decision-making.

Multiple-Choice Questions

1. **What is the primary difference between vagueness and uncertainty?**
   - A) Vagueness deals with missing information; uncertainty deals with fuzzy concepts.
   - B) Vagueness deals with imprecise information; uncertainty deals with missing or incomplete data.
   - C) Vagueness and uncertainty are the same concepts.
2. **What is a fuzzy membership function?**
   - A) A Boolean true/false function.
   - B) A function that assigns degrees of membership to elements.
   - C) A frequentist probability measure.
3. **Which paradigm interprets probabilities as subjective beliefs?**
   - A) Frequentism
   - B) Bayesianism
   - C) Logical determinism

1. B
- Explanation: Vagueness refers to unclear boundaries, while uncertainty arises from incomplete or missing data.
2. B
- Explanation: Fuzzy membership functions normalize values between 0 and 1 to describe the degree of belonging.
3. B
- Explanation: Bayesianism expresses probabilities as degrees of belief about an event.

Glossary of Terms

- **Fuzzy Set**: A set where elements have degrees of membership (0 to 1).
  - Example: “Tall” with values like 0.7 or 0.9.
- **Uncertainty**: Lack of complete information.
- **Atomic Event**: The smallest outcome in a sample space.
  - Example: Rolling a 3 on a die.
- **Random Variable**: A function assigning values to atomic events.
  - Example: Odd/Even outcomes for a die roll.

Tips for Remembering Key Concepts

- Memorize **key differences**: Vagueness (fuzzy logic) vs. Uncertainty (probability).
- Relate fuzzy logic to everyday vague terms (e.g., "warm" or "tall").

L9: Uncertainty (Probabilities) and Bayesian Networks

Tuesday, December 17, 2024 7:13 PM

Introduction to Probability

The lecture focuses on the use of **probabilities** and **Bayesian networks** to model uncertainty in AI systems. It explains fundamental concepts of probability, conditional probability, and Bayes' theorem, and demonstrates their practical applications in intelligent systems.

Key Sections

1. Basic Concepts of Probability

- **Sample Space (Ω)**: The set of all possible outcomes of an experiment.
  - Example: Coin toss → {Heads, Tails}.
- **Atomic Events**: Individual outcomes within the sample space.
  - Example: Rolling a dice → Each number {1, 2, 3, 4, 5, 6}.
- **Events**: Combinations of atomic events or subsets of the sample space.
- **Probability Space**: A sample space transformed with assigned probabilities.
  - Probabilities range between **0 (impossible)** and **1 (certain)**.
  - Total probability across all atomic events equals **1**.

2. Random Variables

A **random variable** is a function assigning values (binary, discrete, or continuous) to outcomes:

- Example: For the random variable Odd with range {True, False}, Odd(1) = True, Odd(2)=False.

Probability Distribution:

- Describes the likelihood of all possible values of a random variable.
- Example: P(X=x) is the sum of probabilities for all atomic events where X=x.

Joint Probability Distribution:

- The probability of two or more random variables occurring simultaneously.
  - Example: Probability of pulling a "heart" card valued greater than 4.

3. Conditional Probability

- Represents the likelihood of an event occurring given that another event has occurred.
  - Formula:
    $$P(A|B) = \frac{P(A \cap B)}{P(B)}$$
  - Example: Rolling a 6 (A) given an even roll (B) → $\frac{\left(\frac{1}{6}\right)}{\left(\frac{1}{2}\right)} = \frac{1}{3}$

Bayes' Theorem:

- Used to calculate reverse probabilities (inversion problems).
- Formula: $P(A|B) = \frac{P(B|A) \cdot P(A)}{P(B)}$

Example:

- Probability of having COVID (C) given a positive test result (+):
- $P(C|+) = \frac{P(+|C) \cdot P(C)}{P(+)}$

- Result demonstrates the importance of **prior probability** in decision-making.

4. Bayesian Networks

A **Bayesian network** (BN) is a probabilistic graphical model that represents conditional dependencies between variables:

- **Nodes**: Represent random variables.
- **Edges**: Represent dependencies.
- **Independence**:
  - A and B are independent if $P(A|B) = P(A)$.
  - **Conditional Independence**: If $P(A|B,C) = P(A|C)$, then A is conditionally independent of B given C.

Advantages of Bayesian Networks:

- Reduces data requirements by assuming conditional independence.
- Simplifies calculations of joint probabilities using the **chain rule**.

5. Chain Rule for Probabilities

The chain rule simplifies joint probability distributions by factoring them into conditional probabilities: $P(A,B,C) = P(A|B,C) \cdot P(B|C) \cdot P(C)$

- Example: In an intelligent system, **"Passing"** depends on **"Understanding"**, which in turn depends on attending lectures or workgroups.

Key Takeaways

1. **Sample Space**: Foundation of probability calculations.
2. **Random Variables**: Functions that map outcomes to values.
3. **Conditional Probability**: Key to reasoning under uncertainty.
4. **Bayes' Theorem**: Powerful for reversing known probabilities.
5. **Bayesian Networks**: Graphical models that simplify complex dependencies.

Practical Applications

- **Medical Diagnosis**: Calculating disease probabilities based on observed symptoms.
- **Spam Detection**: Estimating the likelihood of spam given specific keywords.
- **Autonomous Systems**: Using Bayesian networks to predict system behavior under uncertainty.

Multiple-Choice Questions

1. **What is an atomic event in probability?**
   - A) An event that is impossible.
   - B) An individual outcome in a sample space.
   - C) A combination of multiple events.
2. **What does Bayes' Theorem calculate?**
   - A) Probability of an event given a related event.
   - B) Frequency of outcomes in a sample space.
   - C) Probability of mutually exclusive events.
3. **What is a Bayesian Network?**
   - A) A deterministic system for calculating outcomes.
   - B) A graphical model representing dependencies between variables.
   - C) A sample space for Boolean logic.

1. B

- Explanation: Atomic events are the smallest indivisible outcomes.

2. A
   - Explanation: Bayes' Theorem reverses conditional probabilities.
3. B
   - Explanation: Bayesian networks model conditional dependencies using nodes and edges.

Glossary of Terms

- **Sample Space**: All possible outcomes of an experiment.
  Example: Rolling a die → {1, 2, 3, 4, 5, 6}.
- **Conditional Probability**: Likelihood of an event given another event.
- **Bayes' Theorem**: Formula for reversing conditional probabilities.
- **Joint Probability**: Probability of two or more events occurring together.
- **Bayesian Network**: Probabilistic model for conditional dependencies.

L10: Machine Learning 1

Tuesday, December 17, 2024 8:37 PM

Introduction

This lecture introduces key concepts of **machine learning (ML)**, focusing on **classification tasks** and foundational algorithms like the **Naive Bayes Classifier** and **k-Nearest Neighbors (k-NN)**. The lecture also outlines various ML problem types, learning paradigms, and applications across supervised and unsupervised learning.

Part 1: Classification

1.1 Overview of Classification

- **Definition**: Classification assigns a **class label** to a data point based on input features.
- **Examples**:
  - Predicting if a student will pass a course based on lecture and workgroup attendance.
  - Binary, multiclass, and multilabel classification problems.

Variant	Description	Example
Binary Classification	Two classes	Pass vs. Fail
Multiclass Classification	More than two classes	Grade categories: A, B, C, D
Multilabel Classification	Multiple labels per data point	Document topic classification

1.2 Naive Bayes Classifier

Bayesian Classifier

- **Goal**: Find the class C with the highest probability given features F1, F2, ..., Fn.
- Formula based on **Bayes' Theorem**: $P\left(C|F_{1,\ldots},F_n\right) = \frac{P(C)\cdot P\left(F_1,\ldots,F_n|C\right)}{P\left(F_1,\ldots,F_n\right)}$

Conditional Independence Assumption

To simplify calculations, Naive Bayes assumes that features are **conditionally independent** given the class: $P\left(F_i|C,F_j\right) = P\left(F_i|C\right)$

This results in: $P\left(C,F_{1,\ldots},F_n\right) = P(C)\cdot \Pi P\left(F_i|C\right)$

- The classifier is "naive" because this assumption rarely holds perfectly.

Naive Bayes Steps:

1. **Preprocessing**: Choose features, split data into training and test sets.
2. **Training**: Estimate prior and conditional probabilities.
3. **Testing**: For a new data point, compute the probabilities for each class and pick the most likely.

Example:

Classifying whether a sentence belongs to "Programming" or "Intelligent Systems":

- Features: Keywords like *Python, Algorithm, Probability, File*.
- Compute P(C|features) for each class.

1.3 Smoothing in Naive Bayes

To handle zero probabilities:

- **Laplace Smoothing**: Assign small probabilities (e.g., 0.01) to unseen features.

1.4 k-Nearest Neighbor (k-NN)

Definition

- k-NN predicts a class label based on the **majority class** of its **k nearest neighbors** in feature space.

Steps:

1. Measure distances between a new data point and labeled examples.
2. Find the **k closest neighbors**.
3. Assign the class label based on the majority vote.

Example:

- Classifying a text into categories like "AI", "Law", or "Chemistry."

k-NN Challenges:

- Sensitive to **noise** in the data.
- Performance depends on the choice of k.

Part 2: Introduction to Machine Learning

2.1 Adaptive Intelligent Agents

- **Definition**: Agents that learn from interactions with the environment or other agents.
- **Learning**:
  - Modifies decision-making mechanisms.
  - Essential for operating in unknown environments.

2.2 Types of Machine Learning

1. **Supervised Learning**:
   - Training data includes labeled examples.
   - Example: Classification, Regression.
2. **Unsupervised Learning**:
   - No labels in training data.
   - Example: Clustering, Dimensionality Reduction.
3. **Semi-Supervised Learning**:
   - Combines labeled and unlabeled data.
   - Example: Active learning (labeling high-impact data).
4. **Reinforcement Learning**:
   - Learning through rewards and punishments.

2.3 Types of ML Tasks

Task	Description	Example
Classification	Assigning labels to data points.	Handwriting recognition (MNIST).
Regression	Predicting numeric values.	Stock market prediction.
Ranking	Sorting items based on criteria.	Search engine results.
Collaborative Filter	Recommending items based on user behavior.	Netflix recommender system.
Clustering	Grouping similar data points.	Document categorization.

Key Takeaways

- **Classification** is a supervised learning problem that assigns labels to data points.
- **Naive Bayes** is simple yet effective, even with its conditional independence assumption.
- **k-NN** predicts classes based on proximity in feature space.
- ML encompasses diverse tasks like **regression, ranking, and clustering**.

Multiple-Choice Questions

1. **What assumption simplifies the Naive Bayes classifier?**

A) Features are linearly dependent.
B) Features are conditionally independent given the class.
C) Features are independent of the class.

2. **What is the purpose of Laplace smoothing in Naive Bayes?**
   A) To increase conditional probabilities.
   B) To handle zero probabilities for unseen features.
   C) To normalize the feature values.
3. **How does k-NN classify a new data point?**
   A) By using probabilities.
   B) By finding the closest labeled data point(s).
   C) By assuming conditional independence.

1. **Correct Answer**: B
2. **Correct Answer**: B
3. **Correct Answer**: B

Glossary

- **Classification**: Assigning labels to data points.
- **Naive Bayes**: A probabilistic classifier assuming conditional independence.
- **k-Nearest Neighbor (k-NN)**: A non-parametric algorithm that classifies based on nearest neighbors.
- **Smoothing**: Handling zero probabilities in probabilistic models.

Tips for Remembering Key Concepts

1. Use **examples** like spam filtering for classification tasks.
2. Memorize ML task types with real-world examples (e.g., Netflix → Collaborative Filtering).

L11: Machine Learning 2

Tuesday, December 17, 2024 8:37 PM

Introduction

This lecture explores the **methodology of machine learning**, focusing on **text features** and **evaluation metrics** for assessing ML models. It builds on previous discussions of supervised and unsupervised learning, introducing practical techniques for text processing and model evaluation.

Part 1: Recap

1.1 Overview of Machine Learning Tasks

- **Supervised Learning**:
  - Input data is labeled.
  - Examples:
    - **Classification**: Predict class labels (e.g., spam/not spam).
    - **Regression**: Predict numerical values (e.g., stock prices).
    - **Ranking**: Order data items (e.g., web search results).
    - **Collaborative Filtering**: Suggest items based on user similarities.
- **Unsupervised Learning**:
  - Input data is unlabeled.
  - Example: **Clustering** groups similar objects.

Part 2: Machine Learning Methodology

2.1 Key Steps in Supervised ML

1. **Data Splitting**:
   - **Training Set**: Used to train the model (e.g., 60%).
   - **Validation Set**: Fine-tunes hyperparameters (e.g., 20%).
   - **Test Set**: Evaluates the model (e.g., 20%).
2. **Features**:
   - Characteristics or attributes used to make predictions (e.g., email length, keyword frequency).
3. **Experimentation Cycle**:
   - Train the model → Tune parameters → Evaluate performance on test data.

2.2 Feature Space

- In ML, data points are represented as **feature vectors** in a multi-dimensional space.
- Example: A tree dataset with **age** and **height** can be visualized on a 2D plane.

Part 3: Text Features

3.1 Extracting Features from Text

- Text classification (e.g., spam detection) relies on converting text into numerical features:
  1. **Keyword Occurrences**:
     - Represent text as a vector of 1s (word present) and 0s (word absent).
  2. **Similarity Metrics**:
     - **Hamming Distance**: Number of differing bits.
       - Example: [0,1,1] and [1,1,0] → Distance = 2.
     - **Euclidean Distance**: Geometric distance in feature space.

        - Example: $\sqrt{(x_1 - y_1)^2 + (x_2 - y_2)^2}$
    - **Cosine Similarity**: Cosine of the angle between vectors.
        - Measures directional similarity.

3.2 Improving Text Features

- **Removing Noise**:
    - Convert text to lowercase.
    - Remove punctuation.
    - **Stemming**: Reduce words to their base forms (e.g., "learning" → "learn").
- **Stop Words**:
    - Remove uninformative words like "the," "and," "it."
- **TF-IDF** (Term Frequency-Inverse Document Frequency):
    - Assigns importance to words based on frequency within a document relative to the entire corpus.
- **Word Frequency**:
    - Track how often words occur in a text.

Part 4: Machine Learning Evaluation

4.1 Model Evaluation

- **Goal**: Assess performance of ML models using the test set.
- **Cross-Validation**:
    - Iteratively split the dataset into different train-test pairs to measure generalizability.

4.2 Classification Evaluation Metrics

- Use a **confusion matrix** to summarize predictions:
    - **True Positive (TP)**: Correct positive predictions.
    - **True Negative (TN)**: Correct negative predictions.
    - **False Positive (FP)**: Incorrectly predicted positives.
    - **False Negative (FN)**: Missed positives.

Metrics:

- **Accuracy**: (TP + TN)/N
- **Error**: 1 — Accuracy
- **Recall** (Sensitivity): TP/(TP + FN)
- **Precision** (Positive Predictive Value): TP/(TP + FP)

4.3 Regression Evaluation

- Use **Mean Squared Error (MSE)**: MSE = $\sum (f(x_i) - y_i)^2$
    - Measures the average squared difference between predicted and actual values.

4.4 Ranking Evaluation

- **Precision@k**:
    - Fraction of relevant items in the top k recommendations.

4.5 Clustering Evaluation

- **Internal Criterion**:
    - High intra-cluster similarity (points within clusters are close).
    - Low inter-cluster similarity (clusters are distinct).

Key Takeaways

1. **Supervised ML** involves training, validation, and testing using labeled data.
2. **Text Features**:
    - Convert text to numerical vectors (keywords, TF-IDF).

    - Clean and preprocess text to improve performance.
3. **Evaluation**:
    - Use confusion matrices for classification.
    - MSE for regression.
    - Precision@k for ranking.
4. **Cross-Validation** ensures robust performance estimates.

Multiple-Choice Questions

1. **What is the purpose of the validation set in supervised learning?**
    - A) To train the model.
    - B) To fine-tune hyperparameters.
    - C) To evaluate final performance.
2. **Which metric measures the similarity between vectors based on their angle?**
    - A) Hamming Distance
    - B) Euclidean Distance
    - C) Cosine Similarity
3. **What is the formula for Recall?**
    - A) TP/(TP + FP)
    - B) TP/(TP + FN)
    - C) (TP + TN)/N
4. **What does TF-IDF represent?**
    - A) Term Frequency-Document Frequency
    - B) Term Frequency-Inverse Document Frequency
    - C) Total Frequency-Indexed Document Feature

1. **Correct Answer**: B
2. **Correct Answer**: C
3. **Correct Answer**: B
4. **Correct Answer**: B

Glossary of Terms

- **Feature Vector**: Numerical representation of data points.
- **TF-IDF**: Importance score for words based on frequency.
- **Confusion Matrix**: Table summarizing classification performance.
- **MSE (Mean Squared Error)**: Regression error metric.
- **Precision@k**: Fraction of correct top-k recommendations.

Tips for Remembering Key Concepts

1. Use real-world examples:
    - TF-IDF → Keyword importance in search engines.
    - Precision@k → Netflix recommendations.
2. Create **flowcharts** for ML methodology:
    - Data → Feature Extraction → Training → Testing → Evaluation.

L12: Machine Learning 3

Tuesday, December 17, 2024 8:37 PM

Introduction

This lecture continues the exploration of **machine learning (ML)**, focusing on **model class selection**, regression, and optimization techniques. It builds on previous lectures by discussing linear and higher-dimensional classifiers, underfitting/overfitting, and methods like gradient descent for finding optimal models.

Part 1: Recap

Key ML Recipe Steps

1. **Feature Selection**: Define characteristics of data, e.g., word occurrence or sentence length.
2. **Model Class Selection**: Decide the type of model (e.g., linear regression, classifiers).
3. **Search Method**: Choose an optimization technique to find the best model.

Features Revisited

- Features are **generic characteristics** of data, making machine learning applicable across domains.
- Example:

  Tree Age Tree Height Class Label

Part 2: Model Class Selection

2.1 Model Class for Classification

- **Linear Classifiers**:
  - A straight line separates two classes in the feature space.
  - Example:
    - Healthy if ax1 + bx2 + cx3 + dx4 + e > 0
    - Sick if ax1 + bx2 + cx3 + dx4 + e < 0
  - Goal: Find the parameters a,b,c,d,e.

Hyperplane Definition:

- A **hyperplane** is one dimension less than the feature space.
- **Maximum Margin Hyperplane**:
  - Select the hyperplane with the maximum margin between classes.
  - This principle is the basis for **Support Vector Machines (SVMs)**.

Handling Non-Linearly Separable Data:

1. Change the **model class** (e.g., use ellipses or splines).
2. **Feature Engineering**:
   - Create new features from existing ones (e.g., x^2, y^2).
   - Higher dimensions make data linearly separable.

2.2 Model Class for Regression

- **Regression**: Predict a numerical value.
- **Mean Squared Error (MSE)**: MSE = $\frac{\sum\left(y_i - h(x_i)\right)^2}{n}$
  - Minimize the sum of squared distances between predictions and actual values.

Model Selection Process:

1. Construct a **hypothesis function** h that fits the data.

2. **Occam's Razor**:
    - Prefer the simplest hypothesis that explains the data.

Example: Predict height from age.

- Simplest model: Linear function h(x)=sx + b.
- Add more features (e.g., x^2) for curved regression.

Part 3: Optimization Techniques

3.1 Finding the Best Model

- The **optimal model** minimizes the error function: Error(s,b)= $\sum(sx_i + b - y_i)^2$
- **Analytical Solutions**:
    - Solve for s and b using derivatives:
        - Differentiate the error function and set it to 0.

3.2 Gradient Descent

- **Definition**: An iterative optimization algorithm for minimizing error.
- **Steps**:
    1. Start with random parameters s,b.
    2. Compute the gradient (partial derivatives).
    3. Move a small step in the opposite direction of the gradient.
    4. Repeat until convergence.

Gradient Descent Properties:

- No guarantee of finding the global minimum.
- May get stuck in **local minima**.
- Requires careful selection of the **learning rate**.

Stochastic Gradient Descent (SGD):

- Instead of using the entire dataset, use a **random sample** or subset to update parameters.
- **Advantages**:
    - Faster for large datasets.
    - More efficient than batch gradient descent.
- **Disadvantage**:
    - Can still converge to local minima.

Part 4: Overfitting and Underfitting

Underfitting:

- The model is **too simple** and fails to capture the underlying patterns in the data.

Overfitting:

- The model is **too complex**, fitting noise in the training data and failing to generalize.

Solution:

- Adjust the **model complexity** or **dimensionality**:
    - Fewer features → simpler model.
    - More features → complex model (risk of overfitting).

Key Takeaways

1. **Model Class Selection**:
    - Classification: Linear or higher-dimensional hyperplanes.
    - Regression: Minimize MSE to fit the best model.
2. **Occam's Razor**: Choose the simplest hypothesis.
3. **Optimization**:

    - Gradient descent minimizes error iteratively.
    - Stochastic gradient descent improves efficiency for large datasets.
4. **Overfitting** and **Underfitting**:
    - Balance model complexity to improve generalization.

Multiple-Choice Questions

1. **What is the purpose of a hyperplane in classification?**
    - A) To measure error.
    - B) To separate classes in feature space.
    - C) To minimize variance.
2. **What does Occam's Razor recommend?**
    - A) Use the simplest model that fits the data.
    - B) Always use high-dimensional classifiers.
    - C) Select the model with the smallest margin.
3. **What is the key difference between gradient descent and stochastic gradient descent?**
    - A) SGD uses a subset of data for each iteration.
    - B) Gradient descent always finds the global minimum.
    - C) SGD is slower than gradient descent.
4. **What causes overfitting?**
    - A) Too little training data.
    - B) A model that is too simple.
    - C) A model that is too complex.

1. **Correct Answer**: B
2. **Correct Answer**: A
3. **Correct Answer**: A
4. **Correct Answer**: C

Glossary

- **Hyperplane**: A decision boundary in classification problems.
- **Occam's Razor**: Principle that prefers the simplest solution.
- **Gradient Descent**: Iterative method to minimize a loss function.
- **Stochastic Gradient Descent**: Faster version of gradient descent using subsets of data.
- **Overfitting**: Model is too tailored to training data, reducing generalizability.
- **Underfitting**: Model is too simple to capture patterns in the data.

L13: Machine Learning 4

Tuesday, December 17, 2024 8:38 PM

Introduction

This lecture discusses advanced machine learning techniques, focusing on **clustering**, **artificial neural networks (ANNs)**, and the **Generative Pre-trained Transformer (GPT)** model. It provides insights into model selection for clustering, the architecture of neural networks, and the principles behind GPT-based AI systems.

Part 1: Recap

Key ML Concepts:

1. **Model Selection**:
   - **Classification**: Selecting a model to separate data into predefined categories.
   - **Regression**: Fitting a model to predict continuous numerical values.
2. **Consistency and Separation**:
   - Consistent models match observed data without overfitting or underfitting.

Part 2: Model Selection for Clustering

Clustering Overview:

- **Definition**: An unsupervised method to group data points into clusters based on similarity.
- Example: Grouping data points based on **bill length** and **bill depth**.

2.1 k-Means Clustering

- **Process**:
  1. Choose k (number of clusters) and random cluster centers.
  2. Assign each point to the **nearest cluster center**.
  3. Recompute cluster centers as the mean of assigned points.
  4. Repeat until cluster centers stabilize.
- **Example**:
  - Clustering data into k=3 clusters.

Step	Action
Step 1	Randomly initialize cluster centers.
Step 2	Assign points to closest centers.
Step 3	Recompute means for each cluster.
Step 4 (Iterate)	Reassign points and update means iteratively.

2.2 Evaluation of Clustering

- **Silhouette Coefficient**:
  - Measures clustering quality between **-1 and 1**.
  - **High value**: Well-clustered points.
  - **Low value**: Poorly clustered points.
- **Criteria**:
  1. **Intra-cluster similarity**: High similarity within clusters.
  2. **Inter-cluster similarity**: Low similarity between clusters.

Challenges:

- Choosing the best k depends on the problem context and size of clusters.

Part 3: Artificial Neural Networks (ANNs)

3.1 The Perceptron

- Inspired by **biological neurons**.
- **Mathematical Representation**:
  Y = w1x1 + w2x2 + b
  - w: Weights, x: Inputs, b: Bias.
- **Limitations**:
  - Perceptrons are **linear** and cannot model complex, non-linear relationships.

3.2 Adding Non-Linearity

- **Activation Functions**:
  - Functions applied to introduce non-linearity.
  - Example: **Sigmoid Function**: σ(z) = $\frac{1}{1+e^{-z}}$

3.3 Feedforward Neural Networks

- **Architecture**:
  - Input Layer → Hidden Layers → Output Layer.
  - Weights and biases flow **forward** in one direction.
- **Classification Tasks**:
  - **Binary**: Sigmoid activation outputs probabilities.
  - **Multiclass**: Use the **Softmax** function for probabilities across classes.

Softmax:

$$P(C_k|x) = \frac{e^{zk}}{\Sigma_j e^{zj}}$$

3.4 Backpropagation

- **Purpose**: Adjust network weights to minimize error.
- **Steps**:
  1. Calculate **loss function** (error).
  2. Compute gradients of the loss with respect to weights.
  3. Use **(stochastic) gradient descent** to update weights.

Part 4: Deep Learning and Transformers

4.1 Multilayer Perceptrons (MLPs)

- Deep neural networks are composed of **many hidden layers** to learn hierarchical features:
  - **Low-level**: Basic features (e.g., edges in images).
  - **High-level**: Complex features (e.g., objects or faces).

4.2 Transformer Architecture

- **Overview**:
  - **Key Innovation**: Self-attention mechanism allows focusing on relevant parts of input data.
  - Trained using **stochastic gradient descent**.
  - **Pre-training**: Predict next words in a massive text dataset (~8 terabytes for GPT-3).
- **Beam Search**:
  - Predicts **top-k word sequences** based on probabilities, randomly choosing from the top k outputs.

- **GPT (Generative Pre-trained Transformer)**:
  - **Training**:
    - Step 1: **Pre-training** with unsupervised learning.
    - Step 2: **Fine-tuning** with supervised feedback.

Key Takeaways

1. **Clustering**:
   - k-Means clustering groups data into k clusters.
   - Silhouette Coefficient evaluates clustering quality.
2. **Artificial Neural Networks**:
   - Perceptrons are linear but gain non-linearity through activation functions.
   - Feedforward networks handle both classification and regression tasks.
3. **Deep Learning**:
   - Deep networks can model complex patterns using multiple layers.
4. **GPT**:
   - Trained on massive text datasets using **unsupervised** and **supervised fine-tuning** techniques.

Multiple-Choice Questions

1. **What does the Silhouette Coefficient measure?**
   - A) Model accuracy.
   - B) Cluster quality based on similarity.
   - C) Training loss in neural networks.
2. **Why are activation functions important in neural networks?**
   - A) To add non-linearity to the model.
   - B) To reduce model parameters.
   - C) To calculate gradients.
3. **What is the main purpose of backpropagation?**
   - A) To compute loss.
   - B) To adjust weights based on error gradients.
   - C) To add activation functions.
4. **How does GPT generate text sequences?**
   - A) Through gradient descent alone.
   - B) Using Beam Search to predict high-probability words.
   - C) By clustering words into groups.

1. **Correct Answer**: B
2. **Correct Answer**: A
3. **Correct Answer**: B
4. **Correct Answer**: B

Glossary of Terms

- **Clustering**: Grouping data into clusters without labels.
- **k-Means**: A clustering algorithm that iteratively refines cluster centers.
- **Activation Function**: Introduces non-linearity in neural networks (e.g., Sigmoid, ReLU).
- **Feedforward Network**: A type of ANN where data flows in one direction.
- **Backpropagation**: A method for training neural networks by adjusting weights.
- **Transformer**: A deep learning architecture utilizing attention mechanisms.
- **GPT**: Generative Pre-trained Transformer for text generation.